

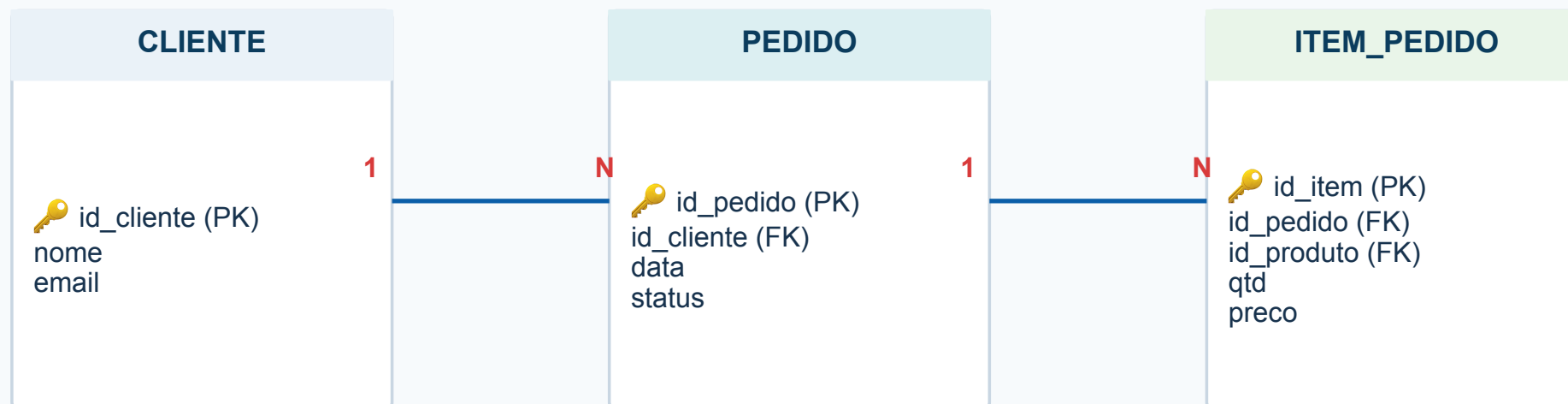
Conceitos essenciais — JOINS: INNER, LEFT, RIGHT e FULL

O que o aluno deve conseguir explicar antes de executar o laboratório.

- INNER JOIN retorna somente correspondências.
- LEFT JOIN mantém todas as linhas da esquerda.
- RIGHT JOIN é o espelho do LEFT, mas normalmente pode ser reescrito invertendo as tabelas.
- FULL OUTER JOIN existe no SQL Server; MySQL exige emulação com UNION de LEFT/RIGHT ou estratégia equivalente.
- Um JOIN correto depende da condição ON correta; JOIN sem relação pode gerar produto cartesiano.

Mapa visual — relacionamentos que viram JOIN

Cliente → Pedido → Item



Regra de leitura: “um CLIENTE pode possuir muitos PEDIDOS; um PEDIDO possui muitos ITENS”.

As chaves estrangeiras ficam no lado N da relação.

Demonstração guiada — comparação entre SGBDs

Execute em banco de laboratório e destaque somente as diferenças de dialeto.

SQL SERVER

```
SELECT c.nome,p.id_pedido,p.total
FROM Cliente c
INNER JOIN Pedido p ON
p.id_cliente=c.id_cliente;
```

```
SELECT c.nome,p.id_pedido
FROM Cliente c
LEFT JOIN Pedido p ON
p.id_cliente=c.id_cliente;
```

```
SELECT a.id,b.id FROM A a
FULL OUTER JOIN B b ON b.id=a.id;
```

MYSQL

```
SELECT c.nome,p.id_pedido,p.total
FROM Cliente c
INNER JOIN Pedido p ON
p.id_cliente=c.id_cliente;
```

```
SELECT c.nome,p.id_pedido
FROM Cliente c
LEFT JOIN Pedido p ON
p.id_cliente=c.id_cliente;
```

```
-- FULL emulado
SELECT a.id,b.id FROM A a LEFT JOIN B b
ON b.id=a.id
UNION
SELECT a.id,b.id FROM A a RIGHT JOIN B b
ON b.id=a.id;
```

Observação: FULL OUTER JOIN é nativo no SQL Server; no MySQL é preciso emular.

Laboratório guiado

Aula 7

Checklist de experimentação — execute, observe e explique o resultado

- 1 Comparar INNER e LEFT JOIN usando cliente sem pedido.
- 2 Adicionar terceira tabela e consultar Cliente → Pedido → Item.
- 3 Provocar produto cartesiano removendo o ON e calcular por que a quantidade cresce.
- 4 Buscar registros sem correspondência com LEFT JOIN + WHERE tabela_direita.id IS NULL.
- 5 Discutir por que FULL OUTER JOIN requer adaptação no MySQL.

Exercício BÁSICO — Biblioteca — livros e autores

Aula 7

Objetivo: Praticar INNER JOIN com duas tabelas.

CENÁRIO

A biblioteca quer listar cada livro acompanhado do nome de seu autor.

O QUE FAZER — siga exatamente nesta ordem

- 1 Execute o código base.
- 2 Observe a FK id_autor em LivroBiblioteca.
- 3 Faça INNER JOIN entre LivroBiblioteca e AutorBiblioteca.
- 4 Mostre titulo e nome_autor.
- 5 Use aliases l e a para facilitar a leitura.

RESULTADO ESPERADO

Devem aparecer 4 livros, todos com autor correspondente.

Código base — BÁSICO: Biblioteca — livros e autores

Aula 7

PASSO 1: copie e execute este código antes de resolver. O código apenas prepara os dados; ele não contém a resposta.

SQL SERVER — CÓDIGO BASE

```
DROP TABLE IF EXISTS LivroBiblioteca;
DROP TABLE IF EXISTS AutorBiblioteca;
CREATE TABLE AutorBiblioteca(id_autor
INT PRIMARY KEY,nome_autor
VARCHAR(80));
CREATE TABLE LivroBiblioteca(id_livro
INT PRIMARY KEY,titulo
VARCHAR(100),id_autor INT,FOREIGN
KEY(id_autor) REFERENCES
AutorBiblioteca(id_autor));
INSERT INTO AutorBiblioteca
VALUES(1,'Ana Torres'),(2,'Bruno
Reis'),(3,'Carla Melo');
INSERT INTO LivroBiblioteca
VALUES(10,'Banco de Dados',1),
(11,'Redes',2),(12,'Algoritmos',1),
(13,'UX Básico',3);
```

MYSQL — CÓDIGO BASE

```
DROP TABLE IF EXISTS LivroBiblioteca;
DROP TABLE IF EXISTS AutorBiblioteca;
CREATE TABLE AutorBiblioteca(id_autor
INT PRIMARY KEY,nome_autor
VARCHAR(80));
CREATE TABLE LivroBiblioteca(id_livro
INT PRIMARY KEY,titulo
VARCHAR(100),id_autor INT,FOREIGN
KEY(id_autor) REFERENCES
AutorBiblioteca(id_autor));
INSERT INTO AutorBiblioteca
VALUES(1,'Ana Torres'),(2,'Bruno
Reis'),(3,'Carla Melo');
INSERT INTO LivroBiblioteca
VALUES(10,'Banco de Dados',1),
(11,'Redes',2),(12,'Algoritmos',1),
(13,'UX Básico',3);
```

DICA: No ON, ligue PK da tabela pai com FK da tabela filha.

Gabarito — BÁSICO: Biblioteca — livros e autores

Aula 7

Confira somente depois de tentar. Compare a lógica, não apenas se “rodou sem erro”.

SQL SERVER — SOLUÇÃO

```
SELECT l.titulo,  
a.nome_autor  
FROM LivroBiblioteca l  
INNER JOIN  
AutorBiblioteca a  
    ON a.id_autor =  
l.id_autor;
```

MYSQL — SOLUÇÃO

```
SELECT l.titulo,  
a.nome_autor  
FROM LivroBiblioteca l  
INNER JOIN  
AutorBiblioteca a  
    ON a.id_autor =  
l.id_autor;
```

POR QUE FUNCIONA INNER JOIN retorna somente linhas que possuem correspondência nos dois lados.

Exercício INTERMEDIÁRIO — Loja — clientes com e sem pedidos

Aula 7

Objetivo: Praticar LEFT JOIN e identificar ausência de relacionamento.

CENÁRIO O comercial quer listar todos os clientes, inclusive os que nunca fizeram pedido.

O QUE FAZER — siga exatamente nesta ordem

- 1 Execute o código base.
- 2 Faça LEFT JOIN partindo de ClienteLoja.
- 3 Mostre nome_cliente e id_pedido.
- 4 Observe que clientes sem pedido recebem NULL em id_pedido.
- 5 Acrescente uma segunda consulta que retorne apenas os clientes sem pedidos.

**RESULTADO
ESPERADO**

A primeira consulta deve listar todos os clientes. • A segunda deve retornar apenas clientes cujo id_pedido ficou NULL.

Código base — INTERMEDIÁRIO: Loja — clientes com e sem pedidos

PASSO 1: copie e execute este código antes de resolver. O código apenas prepara os dados; ele não contém a resposta.

SQL SERVER — CÓDIGO BASE

```
DROP TABLE IF EXISTS PedidoLoja;
DROP TABLE IF EXISTS ClienteLoja;
CREATE TABLE
ClienteLoja(id_cliente INT
PRIMARY KEY,nome_cliente
VARCHAR(80));
CREATE TABLE PedidoLoja(id_pedido
INT PRIMARY KEY,id_cliente
INT,valor DECIMAL(10,2),FOREIGN
KEY(id_cliente) REFERENCES
ClienteLoja(id_cliente));
INSERT INTO ClienteLoja
VALUES(1,'Ana'),(2,'Bruno'),
(3,'Carla'),(4,'Diego');
INSERT INTO PedidoLoja
VALUES(100,1,250),(101,1,90),
(102,3,500);
```

MYSQL — CÓDIGO BASE

```
DROP TABLE IF EXISTS PedidoLoja;
DROP TABLE IF EXISTS ClienteLoja;
CREATE TABLE
ClienteLoja(id_cliente INT
PRIMARY KEY,nome_cliente
VARCHAR(80));
CREATE TABLE PedidoLoja(id_pedido
INT PRIMARY KEY,id_cliente
INT,valor DECIMAL(10,2),FOREIGN
KEY(id_cliente) REFERENCES
ClienteLoja(id_cliente));
INSERT INTO ClienteLoja
VALUES(1,'Ana'),(2,'Bruno'),
(3,'Carla'),(4,'Diego');
INSERT INTO PedidoLoja
VALUES(100,1,250),(101,1,90),
(102,3,500);
```

DICA: Para encontrar “sem relacionamento”, LEFT JOIN + IS NULL é um padrão importante.

Gabarito — INTERMEDIÁRIO: Loja — clientes com e sem pedidos

Confira somente depois de tentar. Compare a lógica, não apenas se “rodou sem erro”.

SQL SERVER — SOLUÇÃO

```
SELECT c.nome_cliente, p.id_pedido  
FROM ClienteLoja c  
LEFT JOIN PedidoLoja p ON  
p.id_cliente=c.id_cliente;
```

```
SELECT c.nome_cliente  
FROM ClienteLoja c  
LEFT JOIN PedidoLoja p ON  
p.id_cliente=c.id_cliente  
WHERE p.id_pedido IS NULL;
```

MYSQL — SOLUÇÃO

```
SELECT c.nome_cliente, p.id_pedido  
FROM ClienteLoja c  
LEFT JOIN PedidoLoja p ON  
p.id_cliente=c.id_cliente;
```

```
SELECT c.nome_cliente  
FROM ClienteLoja c  
LEFT JOIN PedidoLoja p ON  
p.id_cliente=c.id_cliente  
WHERE p.id_pedido IS NULL;
```

```
SELECT c.nome_cliente, p.id_pedido, 'Com ou sem  
pedido' AS tipo_consulta  
FROM ClienteLoja c  
LEFT JOIN PedidoLoja p ON p.id_cliente =  
c.id_cliente
```

UNION ALL

```
SELECT c.nome_cliente, NULL AS id_pedido,  
'Apenas sem pedido' AS tipo_consulta  
FROM ClienteLoja c  
LEFT JOIN PedidoLoja p ON p.id_cliente =  
c.id_cliente  
WHERE p.id_pedido IS NULL;
```

POR QUE FUNCIONA

LEFT JOIN preserva todas as linhas da tabela da esquerda, mesmo quando não existe linha correspondente.

Exercício AVANÇADO — RH — funcionários e departamentos

Aula 7

Objetivo: Comparar FULL OUTER JOIN nativo do SQL Server com uma emulação no MySQL.

CENÁRIO O RH precisa enxergar departamentos sem funcionários e funcionários eventualmente sem departamento, para identificar inconsistências de cadastro.

O QUE FAZER — siga exatamente nesta ordem

- 1 Execute o código base.
- 2 No SQL Server, escreva FULL OUTER JOIN.
- 3 No MySQL, emule o resultado usando LEFT JOIN + RIGHT JOIN com UNION.
- 4 Mostre id_funcionario, nome_funcionario e nome_departamento.
- 5 Explique por que o mesmo comando não funciona nativamente nos dois SGBDs.

RESULTADO ESPERADO

O resultado deve preservar linhas dos dois lados, mesmo sem correspondência.

Código base — AVANÇADO: RH — funcionários e departamentos

PASSO 1: copie e execute este código antes de resolver. O código apenas prepara os dados; ele não contém a resposta.

SQL SERVER — CÓDIGO BASE

```
DROP TABLE IF EXISTS FuncionarioRH;
DROP TABLE IF EXISTS DepartamentoRH;
CREATE TABLE
DepartamentoRH(id_departamento INT
PRIMARY KEY,nome_departamento
VARCHAR(60));

CREATE TABLE
FuncionarioRH(id_funcionario INT
PRIMARY KEY,nome_funcionario
VARCHAR(80),id_departamento INT NULL);
INSERT INTO DepartamentoRH
VALUES(1,'TI'),(2,'Financeiro'),
(3,'Jurídico');
INSERT INTO FuncionarioRH
VALUES(10,'Ana',1),(11,'Bruno',2),
(12,'Carla',NULL);
```

MYSQL — CÓDIGO BASE

```
DROP TABLE IF EXISTS FuncionarioRH;
DROP TABLE IF EXISTS DepartamentoRH;
CREATE TABLE
DepartamentoRH(id_departamento INT
PRIMARY KEY,nome_departamento
VARCHAR(60));

CREATE TABLE
FuncionarioRH(id_funcionario INT
PRIMARY KEY,nome_funcionario
VARCHAR(80),id_departamento INT NULL);
INSERT INTO DepartamentoRH
VALUES(1,'TI'),(2,'Financeiro'),
(3,'Jurídico');
INSERT INTO FuncionarioRH
VALUES(10,'Ana',1),(11,'Bruno',2),
(12,'Carla',NULL);
```

DICA: Atenção: UNION remove duplicatas; UNION ALL exigiria tratamento adicional das linhas coincidentes.

Gabarito — AVANÇADO: RH — funcionários e departamentos

Aula 7

Confira somente depois de tentar. Compare a lógica, não apenas se “rodou sem erro”.

SQL SERVER — SOLUÇÃO

```
SELECT
f.id_funcionario,f.nome_funcionario
,d.nome_departamento
FROM FuncionarioRH f
FULL OUTER JOIN DepartamentoRH d
ON
d.id_departamento=f.id_departamento
;
```

MYSQL — SOLUÇÃO

```
SELECT
f.id_funcionario,f.nome_funcionario
,d.nome_departamento
FROM FuncionarioRH f
LEFT JOIN DepartamentoRH d ON
d.id_departamento=f.id_departamento
```

UNION

```
SELECT
f.id_funcionario,f.nome_funcionario
,d.nome_departamento
FROM FuncionarioRH f
RIGHT JOIN DepartamentoRH d ON
d.id_departamento=f.id_departamento
;
```

POR QUE FUNCIONA

FULL OUTER JOIN existe no SQL Server. No MySQL, o mesmo efeito pode ser construído unindo LEFT e RIGHT JOIN.

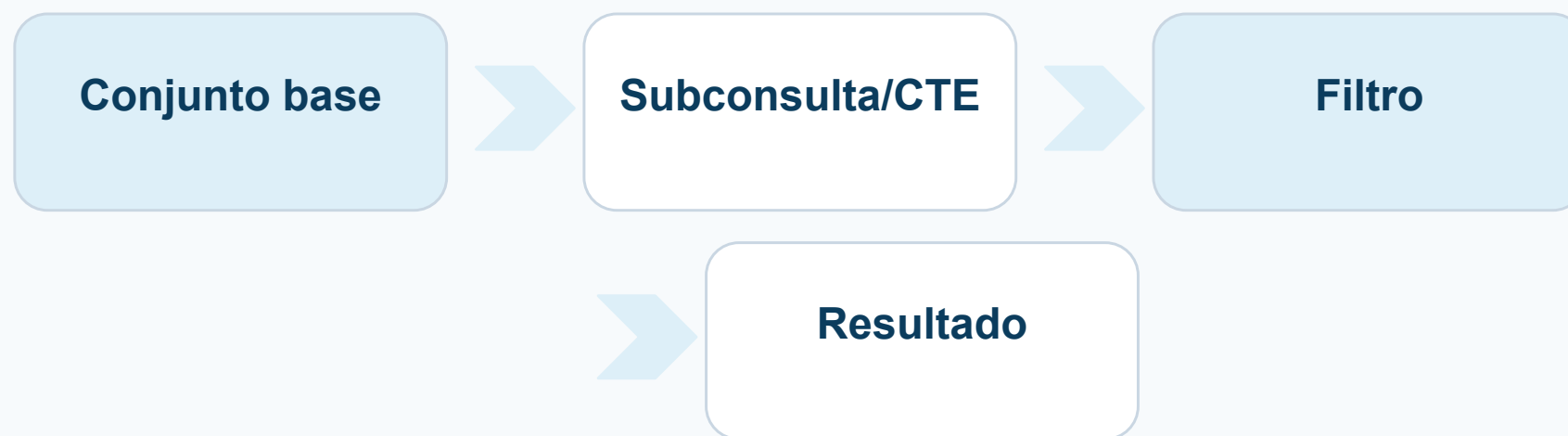
Conceitos essenciais — Subconsultas, EXISTS e CTE

O que o aluno deve conseguir explicar antes de executar o laboratório.

- Subconsulta pode retornar valor escalar, lista ou conjunto usado por outra consulta.
- EXISTS verifica existência e costuma ser natural para perguntas do tipo “há pelo menos um?”.
- CTE (WITH) nomeia uma consulta temporária e melhora legibilidade de consultas complexas.
- Evite subconsulta correlacionada quando um JOIN/agrupamento simples resolver com menor custo e maior clareza.

Mapa visual — consulta em etapas

quebrar problema complexo em conjuntos menores



CTE não é “tabela física”: é uma expressão da instrução atual. Use para legibilidade e composição.

Demonstração guiada — comparação entre SGBDs

Execute em banco de laboratório e destaque somente as diferenças de dialeto.

SQL SERVER

```
WITH Totais AS (  
    SELECT id_cliente, SUM(total)  
    total  
    FROM Pedido GROUP BY  
    id_cliente  
)  
SELECT c.nome, t.total  
FROM Cliente c JOIN Totais t  
ON t.id_cliente=c.id_cliente  
WHERE t.total > (SELECT  
AVG(total) FROM Totais);
```

```
SELECT c.nome FROM Cliente c  
WHERE EXISTS(SELECT 1 FROM  
Pedido p WHERE  
p.id_cliente=c.id_cliente);
```

MYSQL

```
WITH Totais AS (  
    SELECT id_cliente, SUM(total)  
    total  
    FROM Pedido GROUP BY  
    id_cliente  
)  
SELECT c.nome, t.total  
FROM Cliente c JOIN Totais t  
ON t.id_cliente=c.id_cliente  
WHERE t.total > (SELECT  
AVG(total) FROM Totais);
```

```
SELECT c.nome FROM Cliente c  
WHERE EXISTS(SELECT 1 FROM  
Pedido p WHERE  
p.id_cliente=c.id_cliente);
```

Laboratório guiado

Aula 8

Checklist de experimentação — execute, observe e explique o resultado

- 1 Criar subconsulta escalar para comparar preço com média.
- 2 Usar IN com subconsulta para filtrar IDs pertencentes a um conjunto.
- 3 Reescrever uma consulta IN usando EXISTS.
- 4 Criar uma CTE que agregue vendas e depois filtrar a CTE.
- 5 Comparar legibilidade entre consulta aninhada profunda e CTE equivalente.

Exercício BÁSICO — Loja de roupas — acima da média

Aula 8

Objetivo: Usar subconsulta escalar em um filtro.

CENÁRIO

A gerência quer listar produtos cujo preço está acima da média de preços do catálogo.

O QUE FAZER — siga exatamente nesta ordem

- 1 Execute o código base.
- 2 Primeiro execute `SELECT AVG(preco)` e observe a média.
- 3 Depois use essa consulta dentro do `WHERE`.
- 4 Mostre nome e preco dos produtos acima da média.

RESULTADO ESPERADO

A subconsulta deve retornar um único valor: a média.

Código base — BÁSICO: Loja de roupas — acima da média

PASSO 1: copie e execute este código antes de resolver. O código apenas prepara os dados; ele não contém a resposta.

SQL SERVER — CÓDIGO BASE

```
DROP TABLE IF EXISTS ProdutoRoupa;  
CREATE TABLE ProdutoRoupa(id_produto INT  
PRIMARY KEY,nome VARCHAR(80),preco  
DECIMAL(10,2));  
INSERT INTO ProdutoRoupa  
VALUES(1,'Camiseta',60),(2,'Calça',150),  
(3,'Jaqueta',300),(4,'Boné',50),  
(5,'Tênis',240);
```

MYSQL — CÓDIGO BASE

```
DROP TABLE IF EXISTS ProdutoRoupa;  
CREATE TABLE ProdutoRoupa(id_produto INT  
PRIMARY KEY,nome VARCHAR(80),preco  
DECIMAL(10,2));  
INSERT INTO ProdutoRoupa  
VALUES(1,'Camiseta',60),(2,'Calça',150),  
(3,'Jaqueta',300),(4,'Boné',50),  
(5,'Tênis',240);
```

DICA: Teste a subconsulta sozinha antes de colocá-la dentro da consulta maior.

Gabarito — BÁSICO: Loja de roupas — acima da média

Aula 8

Confira somente depois de tentar. Compare a lógica, não apenas se “rodou sem erro”.

SQL SERVER — SOLUÇÃO

```
SELECT nome,preco
FROM ProdutoRoupa
WHERE preco > (SELECT AVG(preco) FROM
ProdutoRoupa);
```

MYSQL — SOLUÇÃO

```
SELECT nome,preco
FROM ProdutoRoupa
WHERE preco > (SELECT AVG(preco) FROM
ProdutoRoupa);
```

POR QUE FUNCIONA

Uma subconsulta escalar devolve um valor único que pode ser comparado com cada linha da consulta externa.

Exercício INTERMEDIÁRIO — Cursos online — alunos que compraram

Aula 8

Objetivo: Usar EXISTS para testar existência de relacionamento.

CENÁRIO A plataforma quer listar apenas alunos que possuem pelo menos uma compra com status PAGO.

O QUE FAZER — siga exatamente nesta ordem

- 1 Execute o código base.
- 2 Na consulta externa, percorra AlunoCurso.
- 3 Dentro de EXISTS, procure CompraCurso do mesmo aluno com status PAGO.
- 4 Retorne apenas id_aluno e nome.

RESULTADO ESPERADO

Cada aluno deve aparecer no máximo uma vez, mesmo que tenha várias compras pagas.

Código base — INTERMEDIÁRIO: Cursos online — alunos que compraram

Aula 8

PASSO 1: copie e execute este código antes de resolver. O código apenas prepara os dados; ele não contém a resposta.

SQL SERVER — CÓDIGO BASE

```
DROP TABLE IF EXISTS CompraCurso;
DROP TABLE IF EXISTS AlunoCurso;
CREATE TABLE AlunoCurso(id_aluno
INT PRIMARY KEY,nome
VARCHAR(80));
CREATE TABLE
CompraCurso(id_compra INT PRIMARY
KEY,id_aluno INT,status
VARCHAR(20),valor DECIMAL(10,2));
INSERT INTO AlunoCurso
VALUES(1,'Ana'),(2,'Bruno'),
(3,'Carla'),(4,'Diego');
INSERT INTO CompraCurso
VALUES(10,1,'PAGO',120),
(11,1,'PAGO',80),
(12,2,'CANCELADO',150),
(13,4,'PAGO',200);
```

MYSQL — CÓDIGO BASE

```
DROP TABLE IF EXISTS CompraCurso;
DROP TABLE IF EXISTS AlunoCurso;
CREATE TABLE AlunoCurso(id_aluno
INT PRIMARY KEY,nome
VARCHAR(80));
CREATE TABLE
CompraCurso(id_compra INT PRIMARY
KEY,id_aluno INT,status
VARCHAR(20),valor DECIMAL(10,2));
INSERT INTO AlunoCurso
VALUES(1,'Ana'),(2,'Bruno'),
(3,'Carla'),(4,'Diego');
INSERT INTO CompraCurso
VALUES(10,1,'PAGO',120),
(11,1,'PAGO',80),
(12,2,'CANCELADO',150),
(13,4,'PAGO',200);
```

DICA: Dentro de EXISTS, SELECT 1 é suficiente porque o valor retornado não é utilizado.

Gabarito — INTERMEDIÁRIO: Cursos online — alunos que compraram

Aula 8

Confira somente depois de tentar. Compare a lógica, não apenas se “rodou sem erro”.

SQL SERVER — SOLUÇÃO

```
SELECT
a.id_aluno, a.nome
FROM AlunoCurso a
WHERE EXISTS (
    SELECT 1 FROM
    CompraCurso c
    WHERE
    c.id_aluno=a.id_aluno
    AND
    c.status='PAGO' );
```

MYSQL — SOLUÇÃO

```
SELECT
a.id_aluno, a.nome
FROM AlunoCurso a
WHERE EXISTS (
    SELECT 1 FROM
    CompraCurso c
    WHERE
    c.id_aluno=a.id_aluno
    AND
    c.status='PAGO' );
```

POR QUE FUNCIONA

EXISTS pergunta “existe ao menos uma linha que atende à condição?” e evita multiplicar linhas da consulta externa.

Exercício AVANÇADO — Vendas regionais — ranking por total

Objetivo: Usar CTE para separar agregação e filtro.

CENÁRIO A diretoria quer calcular o total vendido por região e depois mostrar somente regiões com total acima de R\$ 10.000.

O QUE FAZER — siga exatamente nesta ordem

- 1 Execute o código base.
- 2 Crie uma CTE chamada TotalRegiao.
- 3 Dentro dela, agrupe por regioao e calcule SUM(valor).
- 4 Na consulta externa, filtre total_vendido > 10000.
- 5 Ordene do maior total para o menor.

RESULTADO ESPERADO

A CTE deve funcionar como um resultado temporário nomeado para a consulta seguinte.

Código base — AVANÇADO: Vendas regionais — ranking por total

PASSO 1: copie e execute este código antes de resolver. O código apenas prepara os dados; ele não contém a resposta.

SQL SERVER — CÓDIGO BASE

```
DROP TABLE IF EXISTS VendaRegional;  
CREATE TABLE VendaRegional(id_venda INT  
PRIMARY KEY,regiao VARCHAR(30),valor  
DECIMAL(12,2));  
INSERT INTO VendaRegional  
VALUES(1,'Sul',6000),(2,'Sul',7000),  
(3,'Sudeste',9000),(4,'Sudeste',8000),  
(5,'Norte',4000),(6,'Norte',3000);
```

MYSQL — CÓDIGO BASE

```
DROP TABLE IF EXISTS VendaRegional;  
CREATE TABLE VendaRegional(id_venda INT  
PRIMARY KEY,regiao VARCHAR(30),valor  
DECIMAL(12,2));  
INSERT INTO VendaRegional  
VALUES(1,'Sul',6000),(2,'Sul',7000),  
(3,'Sudeste',9000),(4,'Sudeste',8000),  
(5,'Norte',4000),(6,'Norte',3000);
```

DICA: Leia primeiro a CTE como se fosse uma tabela temporária e depois leia a consulta externa.